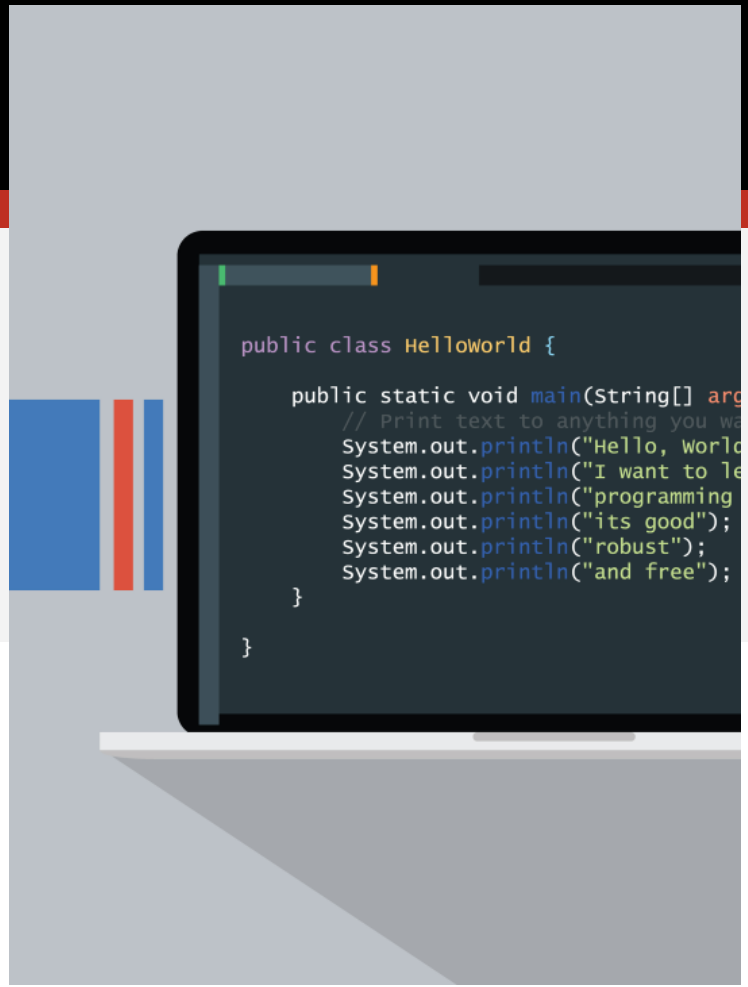
MONTCLAIR
STATE UNIVERSITY

CSITBG2 – Data Structures and Algorithms: Java



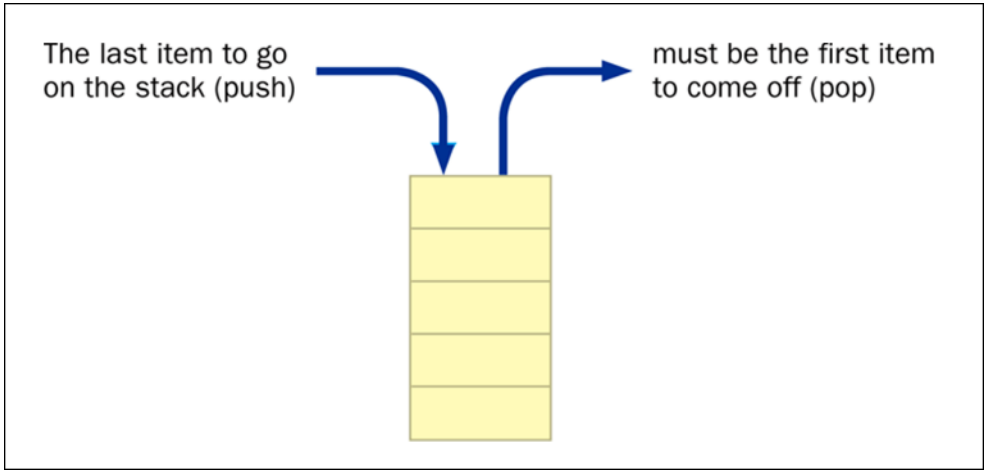
Stack and Queue

Table of Contents

- **Stack and Queue**
- Linked List
- Binary Search Tree I
- Binary Search Tree II and Hash Table

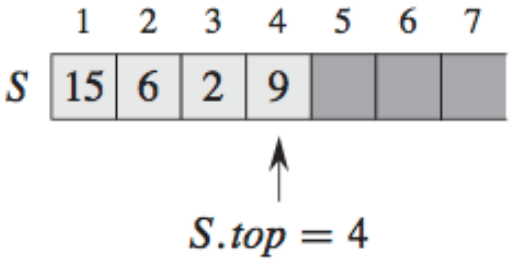
Stack

- A stack is a linear data structure that follows the **Last in, First Out (LIFO)** principle.
  - The last element added to the stack will be the first one to be removed.
  - Analogies: a stack of plates or a stack of books



Stack Attribute

- We can implement a stack of at most n elements with an array S[1…n].
- We have an attribute S.top: the index of the last element.



(a)

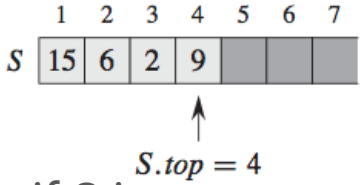
Common Operations

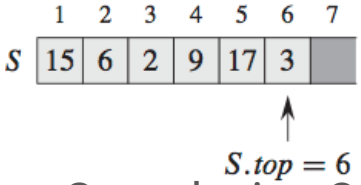
- **Stack-Empty(S):** return TRUE if S is empty. S.top = 0. Time Complexity: O(1)

```
STACK-EMPTY(S)
    if S.top == 0
        return TRUE
    else return FALSE
```

- **Push(S, x):** add x to S, and increment S.top by 1. Time Complexity: O(1)

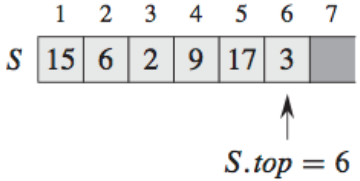
```
PUSH(S,x)
    S.top = S.top + 1
    S[S.top] = x
```

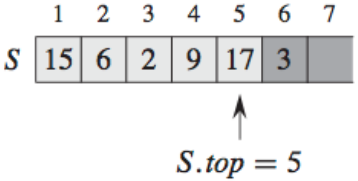




- **Pop(S):** kick out the last element, if S is not empty. Time Complexity: O(1)

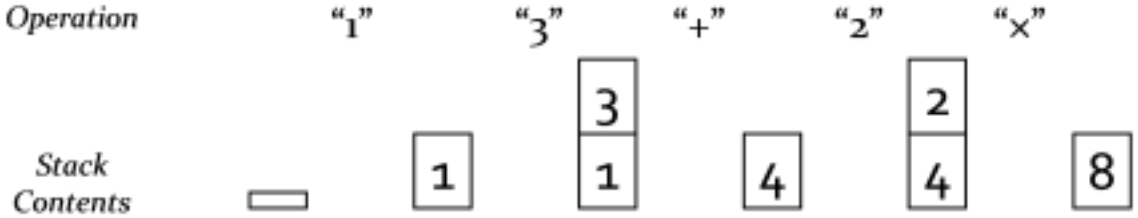
```
STACK-EMPTY(S)
    if S.top == 0
        return TRUE
    else return FALSE
```





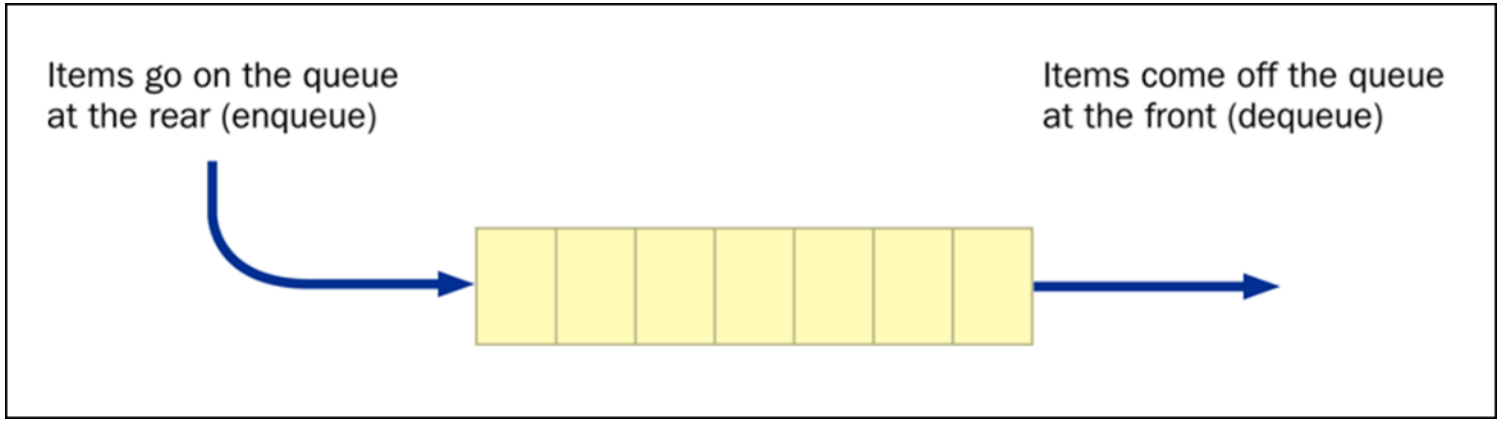
Stack's Application

- Stack is useful in a wide range of applications across software development, algorithms, and systems design.
- Example: application in calculator



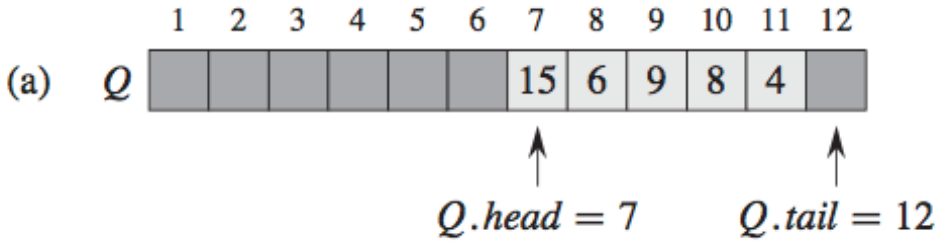
Queue

- A **queue** is a list that adds items only to the rear of the list and removes them only from the front
- It is a FIFO data structure: **First-In, First-Out**
- Analogy: a line of people at a bank teller's window



Queue Attributes

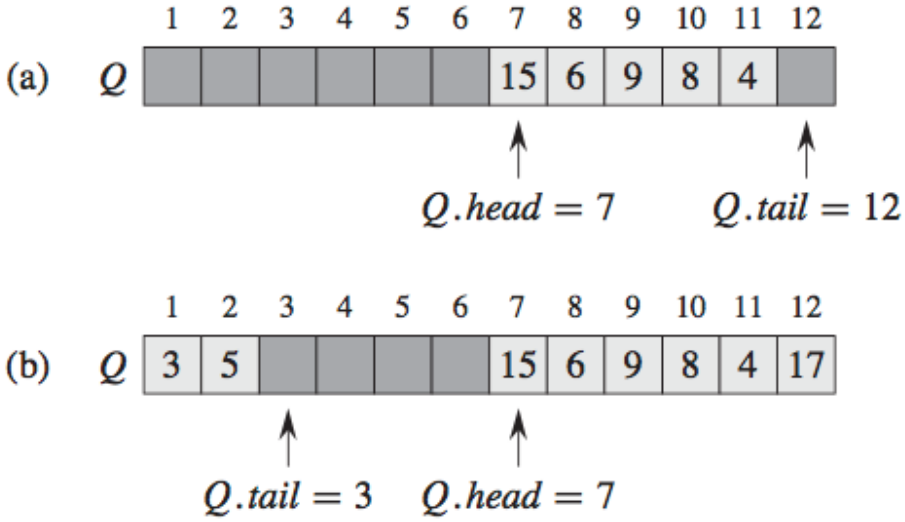
- Queue has two attributes, head and tail
  - **head:** the index of the first element
  - **tail:** the index of the next location where a new element will be inserted
  - When `tail == head` , the queue is empty.
  - When `(tail+1)%length == head` , the queue is full.



Common Operations (1 of 2)

- Two functions in Queue:
  - Enqueue(Q, x): add a new element to Q. Time Complexity: O(1)

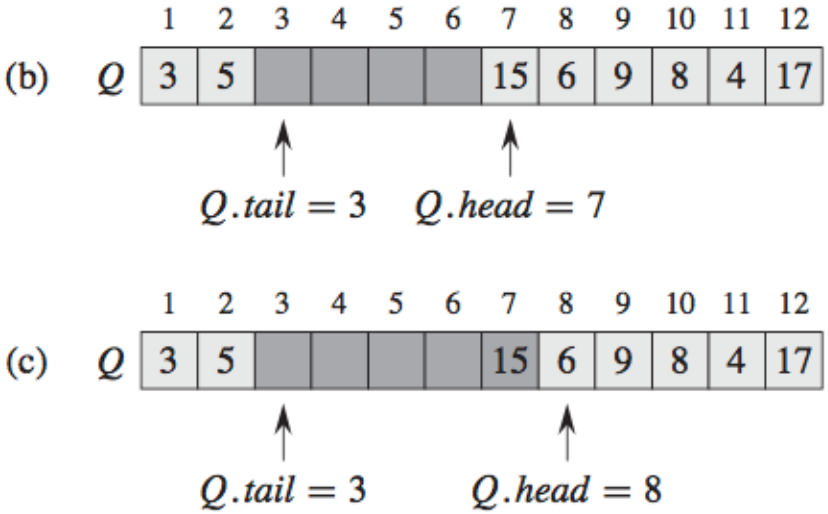
```
ENQUEUE(Q, x)
Q[Q.tail] = x
if Q.tail == Q.length
    Q.tail = 1
else Q.tail = Q.tail + 1
```



Common Operations

- Two functions in Queue:
  - Enqueue(Q, x): add a new element to Q. Time Complexity: O(1)
  - Dequeue(Q): delete the element at head. Time Complexity: O(1)

```
DEQUEUE(Q)
x = Q[Q.head]
if Q.head == Q.length
    Q.head = 1
else Q.head = Q.head + 1
return x
```

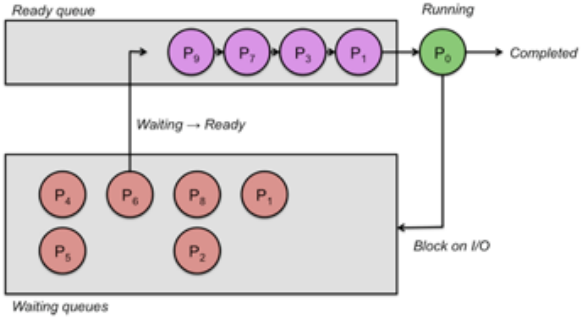


Queue's Application

- Example: application in CPU Scheduling

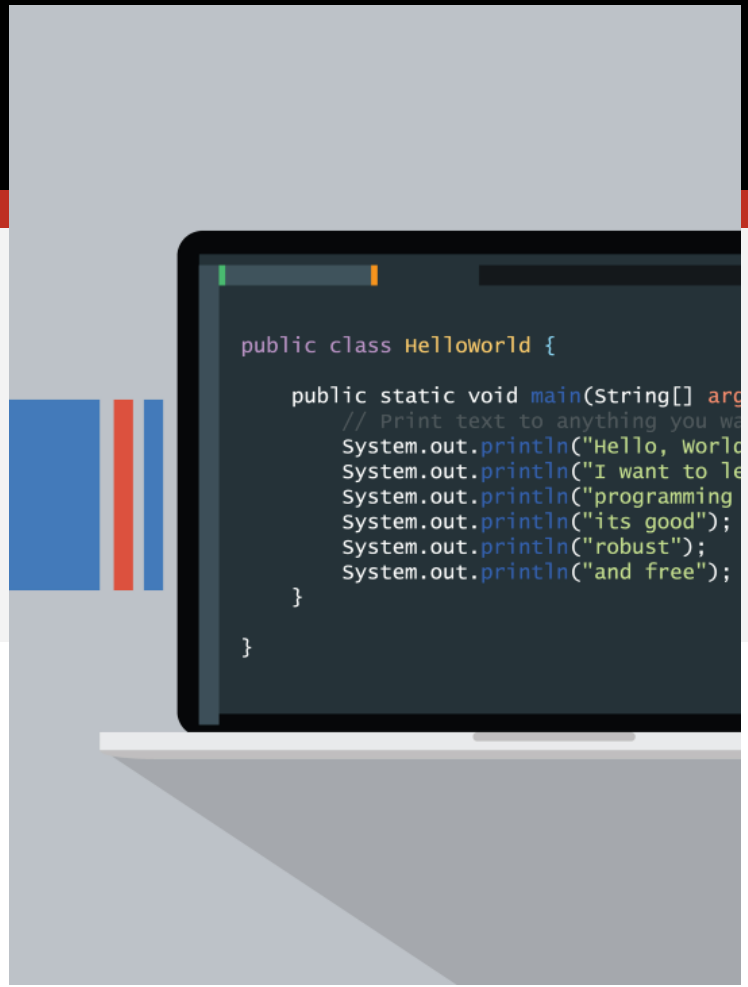
Scheduling Algorithms – First-Come First-Serve Scheduling (FCFS)

- Principle:
  - When a process enters the ready queue, it goes to the tail of the queue.
  - When the CPU is free, it is allocated to the process located in the head of the queue.



MONTCLAIR
STATE UNIVERSITY

CSITBG2 – Data Structures and Algorithms: Java



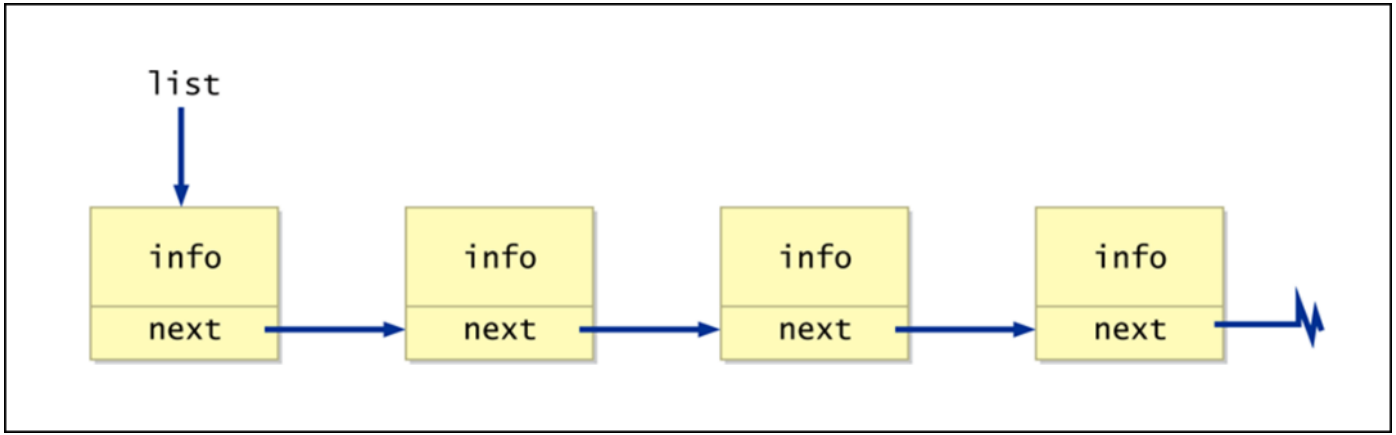
Linked List

Table of Contents

- Stack and Queue
- **Linked List**
- Binary Search Tree I
- Binary Search Tree II and Hash Table

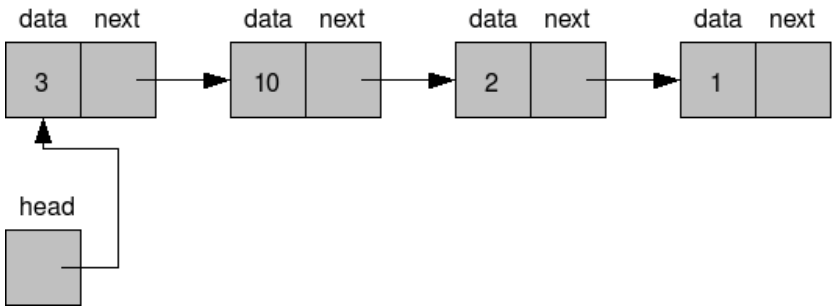
Linked List V.S. Array

- Similarity: objects arranged in a linear order
- Difference:
  - Array: the order is decided by the index
  - Linked List: the order is decided by the pointers

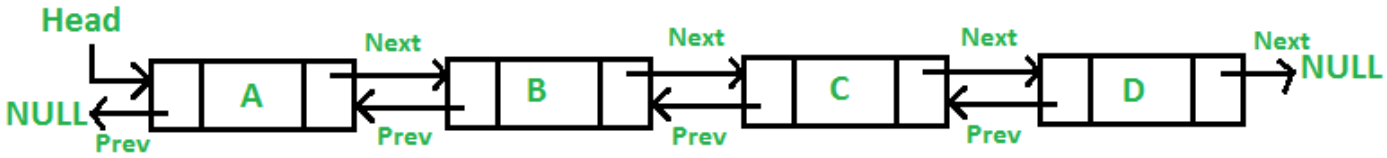


Four Types (1 of 2)

- Singly linked list

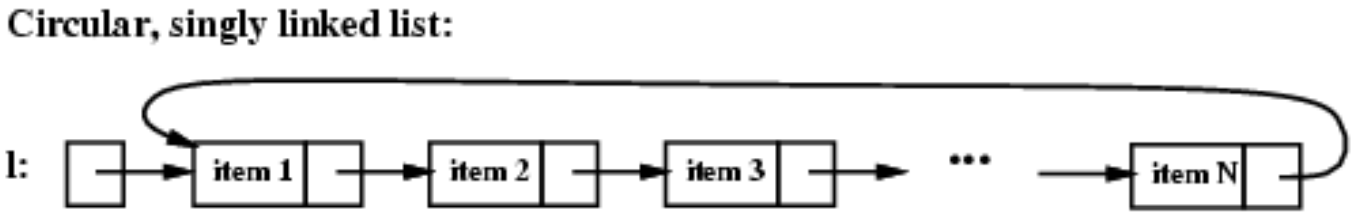


- Doubly linked list

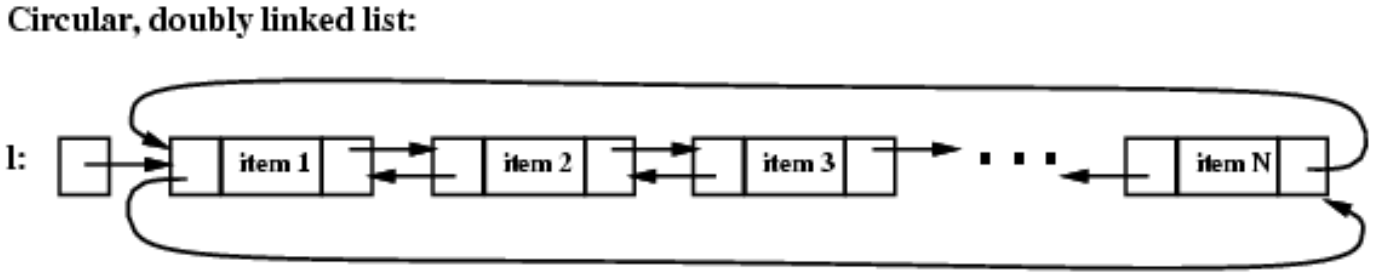


Four Types (2 of 2)

- Circular, singly circular list

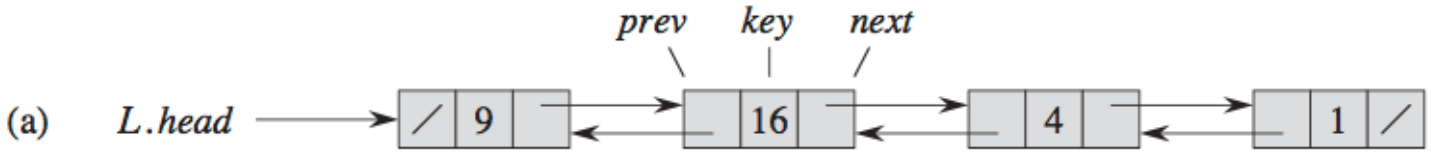


- Circular, doubly circular list



Doubly Linked List

- The doubly linked list has an attribute **head**:
  - Points to the first element of the list
  - If head = NIL, the list is empty.



- Each element of a list is an object with:
  - an attribute **key** that stores the value
  - a pointer **next** that points to its successor
    - If x.next = NIL, the element x is the last element
  - a pointer **prev** that points to its predecessor
    - If x.prev = NIL, the element x is the first element

Common Operations (1 of 3)

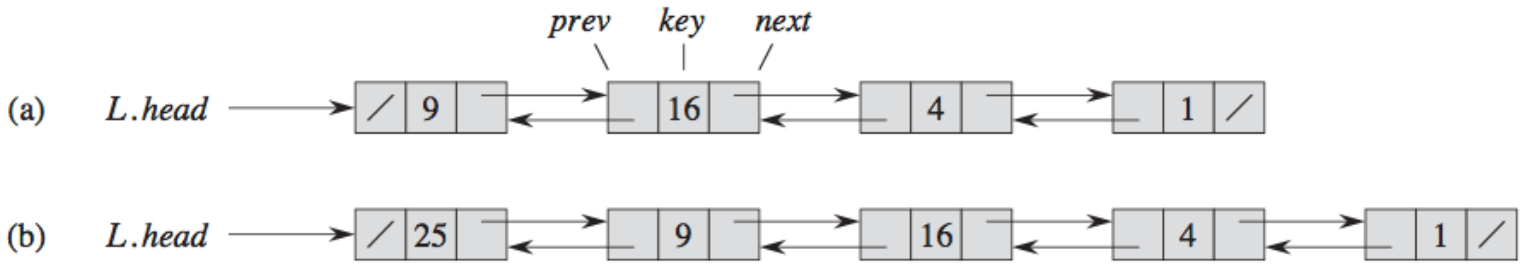
- Linked list supports three functions:
  - List-Search (L, k): finds the first element with key k in the list L.
    - Time complexity: O(n). In the worst case, need to search the entire list.

```
LIST-SEARCH(L, k)
x = L.head
while x ≠ NIL and x.key ≠ k
    x = x.next
return x
```

Common Operations (2 of 3)

- Linked list supports three functions:
  - List-Search (L, k): finds the first element with key k in the list L. O(n)
  - List-Insert (L, x): insert an element x to the front of the list. O(1)

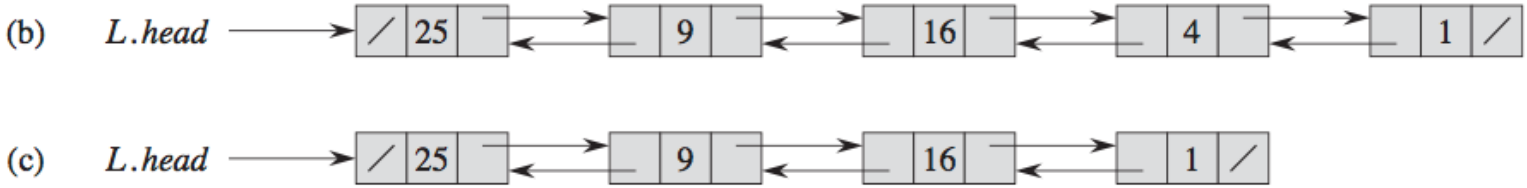
```
LIST-INSERT(L, x)
x.next = L.head
if L.head ≠ NIL
    L.head.prev = x
L.head = x
x.prev = NIL
```



Common Operations (3 of 3)

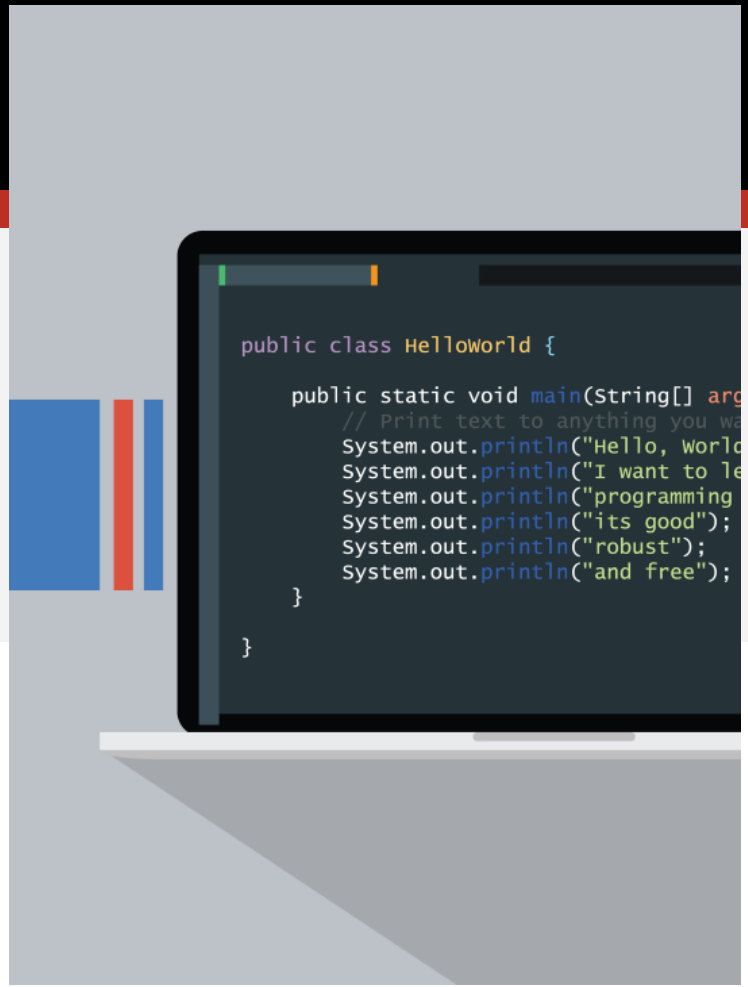
- Linked list supports three functions:
  - List-Search (L, k): finds the first element with key k in the list L. O(n)
  - List-Insert (L, x): insert an element x to the front of the list. O(1)
  - List-Delete (L, x): delete an element x from the list. O(1)

```
LIST-DELETE(L, x)
if x.prev ≠ NIL
    x.prev.next = x.next
else L.head = x.next
if x.next ≠ NIL
    x.next.prev = x.prev
```



MONTCLAIR
STATE UNIVERSITY

CSITBG2 – Data Structures and Algorithms: Java



Binary Search Tree I

Table of Contents

- Stack and Queue
- Linked List
- **Binary Search Tree I**
- Binary Search Tree II and Hash Table

Tree Structure

- In a tree structure,
  - Each node has zero or one parent node.
  - The node that has no parent node is called root.
  - Each node can have zero or multiple children nodes.
  - The nodes that has no child node are called leaf nodes.
  - The node that has no parent node is called root.

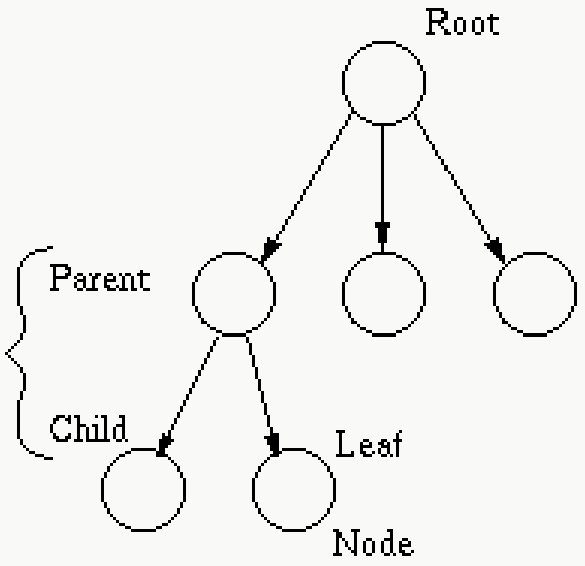
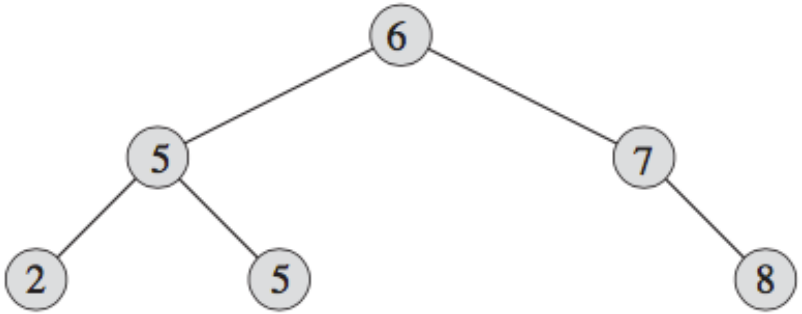


Figure: Tree data structure

Binary Tree

- In a **binary** tree, each node can have **at most two** children nodes.
- One concept: tree height
- Each node hash four attributes:
  - key: the value
  - p: parent node
  - left: left child node
  - right: right child node



Binary Search Tree

- Binary search tree is a special type of binary tree

The keys in a binary search tree are always stored in such a way as to satisfy the ***binary-search-tree property***:

Let x be a node in a binary search tree. If y is a node in the left subtree of x, then $y.key \leq x.key$. If y is a node in the right subtree of x, then $y.key \geq x.key$.

Common Operations

- functions supported by binary search tree:
  - **traverse**: pint out all the keys in a binary search tree
    - **inorder**
    - **preorder**
    - **postorder**
  - **search(k):** search for the first node that stores the value k
  - **minimum**: return the minimum value in a tree
  - **maximum**: return the maximum value in a tree
  - **successor(x)**: return the smallest key that is > x.key
  - **insert(z)**: insert a new node into the tree
  - **delete(z)**: delete a tree node

Traverse

- Inorder tree traversal:
  - print values in the left subtree first
  - then print the key of the root of a subtree
  - last print values in the right subtree

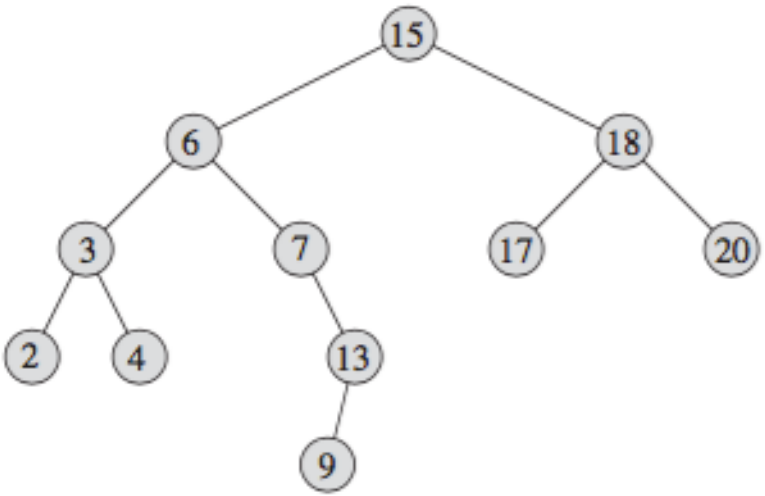
```
INORDER-TREE-WALK(x)
if x ≠ NIL
    INORDER-TREE-WALK(x.left)
    print x.key
    INORDER-TREE-WALK(x.right)
```

- In order to use in-order to traverse a tree, we call *INORDER-TREE-WALK(T.root)*.
- Preorder walk: print the root key before the values in the subtrees.
- Postorder walk: print the root after the values in its subtrees.
- The time complexity is O(n), because we need to visit every node once.

Search

- Input: a key k
- Output: the node with key k if one exists; otherwise, NIL
- Complexity: O(h)

```
TREE-SEARCH(x, k)
if x == NIL or k == x.key
    return x
if k < x.key
    return TREE-SEARCH(x.left, k)
else return TREE-SEARCH(x.right, k)
```



If we search the key 13, we follow the path 15->6->7->13.

Iterative Tree Search

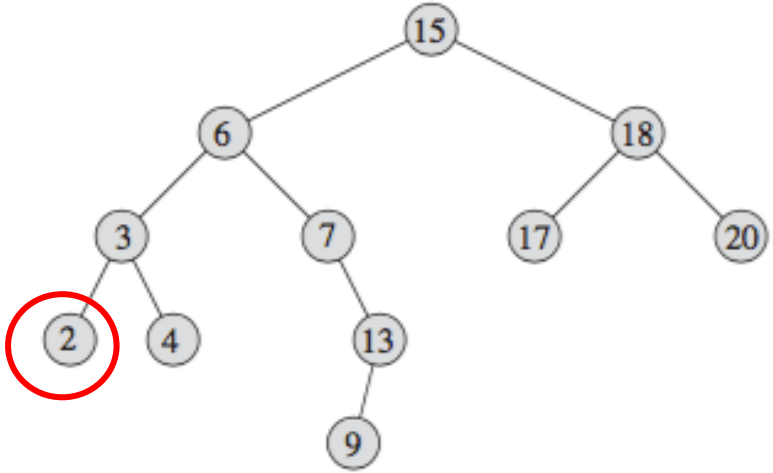
- In theory, we can convert any recursive function into an iterative version.
- Usually, the iterative version runs faster than recursive version.
- An iterative version of the tree search.

```
ITERATIVE-TREE-SEARCH(x, k)
while x ≠ NIL and k ≠ x.key
    if k < x.key
        x = x.left
    else x = x.right
return x
```

Tree Minimum

- Input: a binary search tree node x
- Output: the minimum key in the subtree rooted at x
- Key idea: the leftmost node has the smallest key
- The time complexity is O(h)

```
TREE-MINIMUM(x)
while x.left ≠ NIL
    x = x.left
return x
```

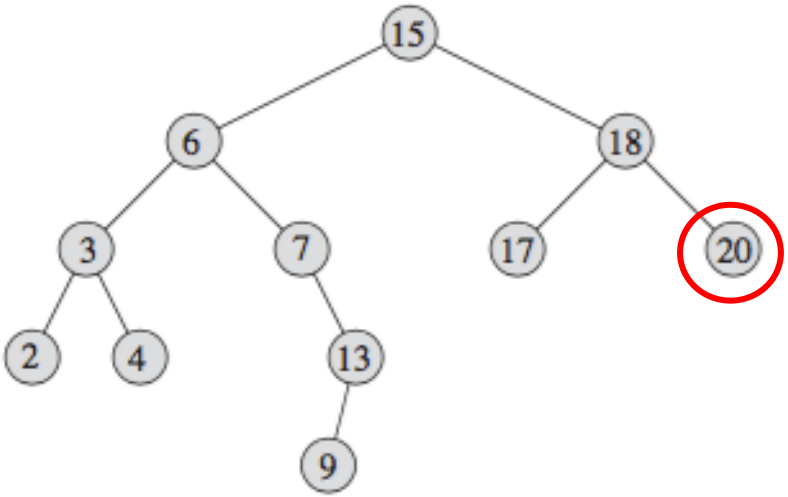


The minimum key in the tree is 2.

Tree Maximum

- Input: a binary search tree node x
- Output: the maximum key in the subtree rooted at x
- Key idea: the rightmost node has the smallest key
- The time complexity is O(h).

```
TREE-MAXIMUM(x)
while x.right ≠ NIL
    x = x.right
return x
```

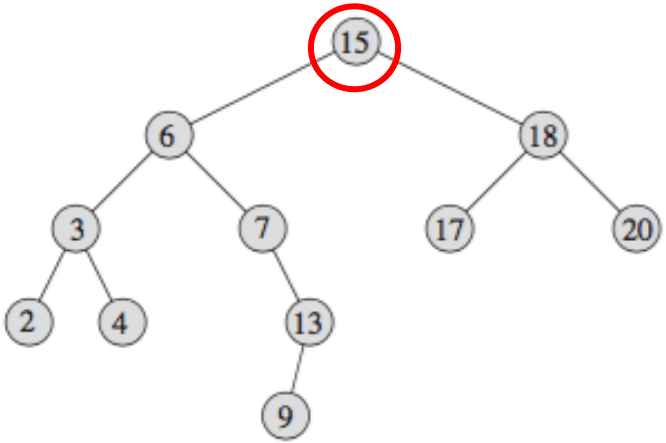


The maximum key in the tree is 20.

Tree Successor

- Input: a tree node
- Output: a tree node that contains the smallest key greater than x.key
- The time complexity is O(h).

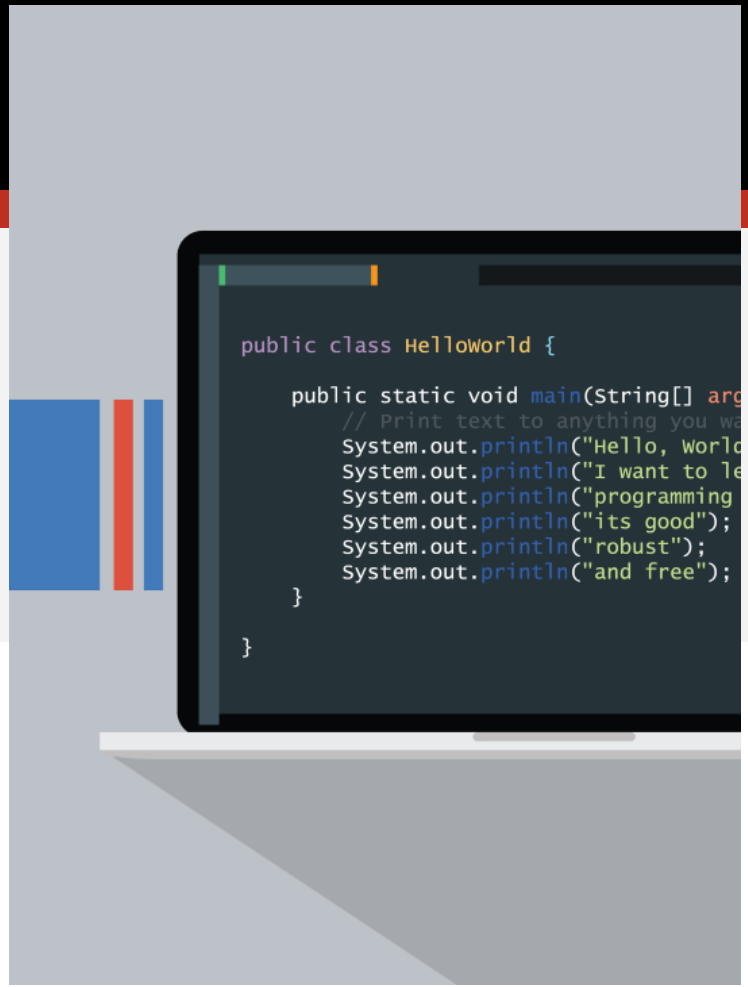
```
TREE-SUCCESSOR(x)
if x.right ≠ NIL
    return TREE-MINIMUM(x.right)
y = x.p
while y ≠ NIL and x == y.right
    x = y
    y = y.p
return y
```



The successor of node 13 in the tree is 15.

MONTCLAIR
STATE UNIVERSITY

CSITBG2 – Data Structures and Algorithms: Java



Binary Search Tree II and Hash Table

Table of Contents

- Stack and Queue
- Linked List
- Binary Search Tree I
- **Binary Search Tree II and Hash Table**

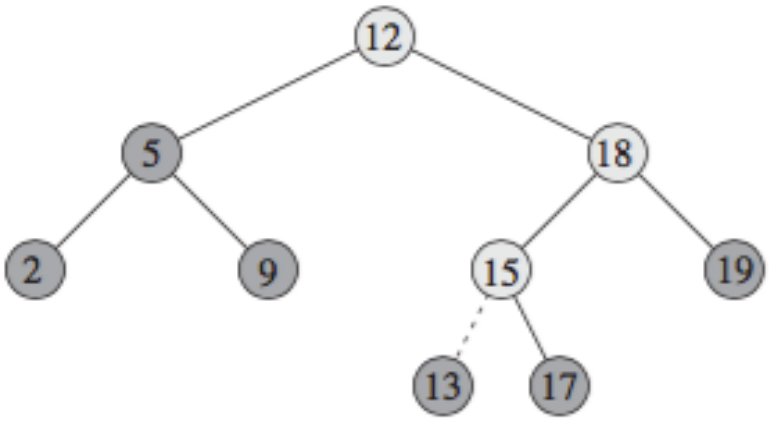
Insert (1 of 2)

- Input: a new node z with z.key
- Output: add z to a tree T
- Key idea: use z.key to find a node y to add z as y's child

```
TREE-INSERT(T, z)
y = NIL
x = T.root
while x ≠ NIL
    y = x
    if z.key < x.key
        x = x.left
    else x = x.right
z.p = y
if y == NIL
    T.root = z          // tree T was empty
elseif z.key < y.key
    y.left = z
else y.right = z
```

Insert (2 of 2)

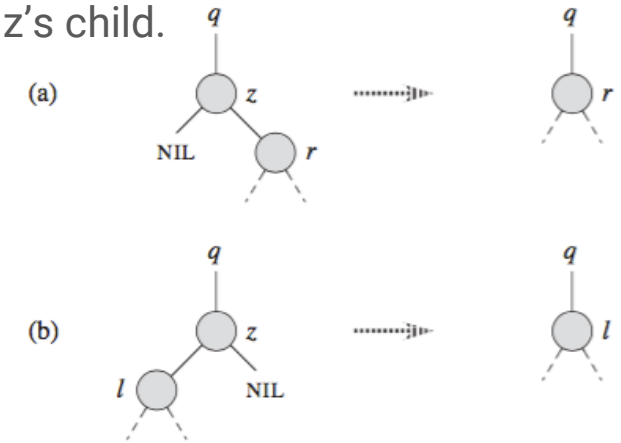
- In the algorithm, we start from the root. In each iteration, we go one level down, until we find an appropriate position to add z.
- Time complexity: O(h)



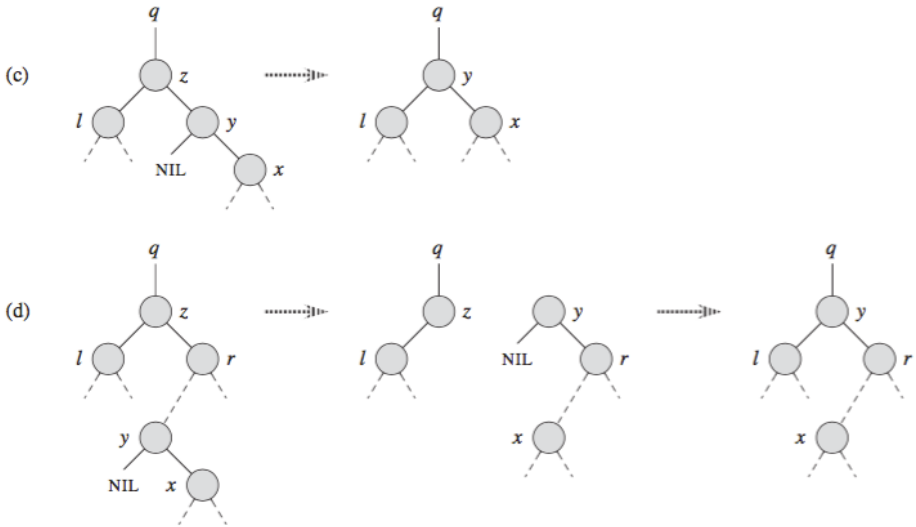
Inserting an item with key 13 into a binary search tree.

Delete (1 of 3)

- If z has no children, then we simply remove it by modifying its parent to replace z with NIL as its child.
- If z has just one child, then we elevate that child to take z's position in the tree by modifying z's parent to replace z by z's child.



- If z has two children, then we find z's successor y and have y take z's position.



Delete (2 of 3)

- Input: a node z in the tree
- Output: remove z from the tree

```
TREE-DELETE(T, z)
if z.left == NIL
    TRANSPLANT(T, z, z.right)
elseif z.right == NIL
    TRANSPLANT(T, z, z.left)
else y = TREE-MINIMUM(z.right)
    if y.p ≠ z
        TRANSPLANT(T, y, y.right)
        y.right = z.right
        y.right.p = y
    TRANSPLANT(T, z, y)
    y.left = z.left
    y.left.p = y
```

```
TRANSPLANT(T, u, v)
if u.p == NIL
    T.root = v
elseif u == u.p.left
    u.p.left = v
else u.p.right = v
if v ≠ NIL
    v.p = u.p
```

Delete (3 of 3)

- In the algorithm, each line takes a constant time, except for the 5-th line, where we call TREE-MINIMUM(z.right) function.
- Time complexity: O(h)

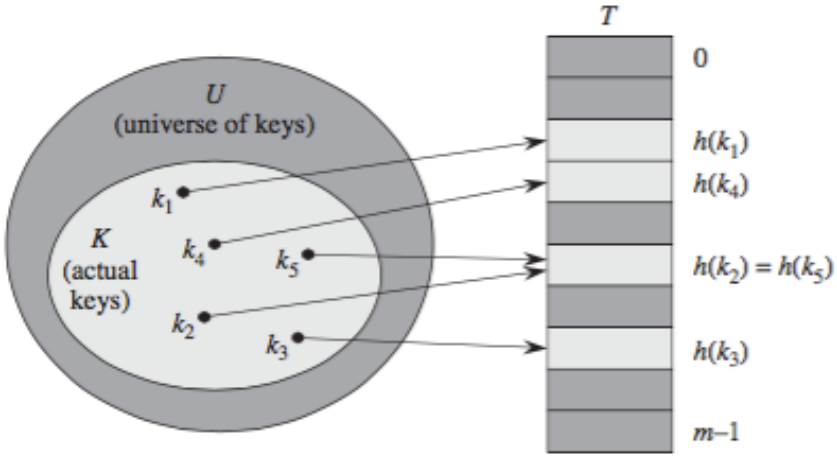
Time Complexity

- What is h?
  - Given values to be randomly inserted into a binary search tree, the expected height is O(lgn).
  - However, there are exceptions.
- In the worst case, the height of a binary search tree can be n.
- This makes the search operation in a binary search tree equivalent to a linked list.
- When all paths from the root to the leaf nodes are equal, the tree is balanced, and the height is minimum, equal to logn.
- We need to reduce the height to be a small value, if possible, as small as logn.

Hash Table (1 of 2)

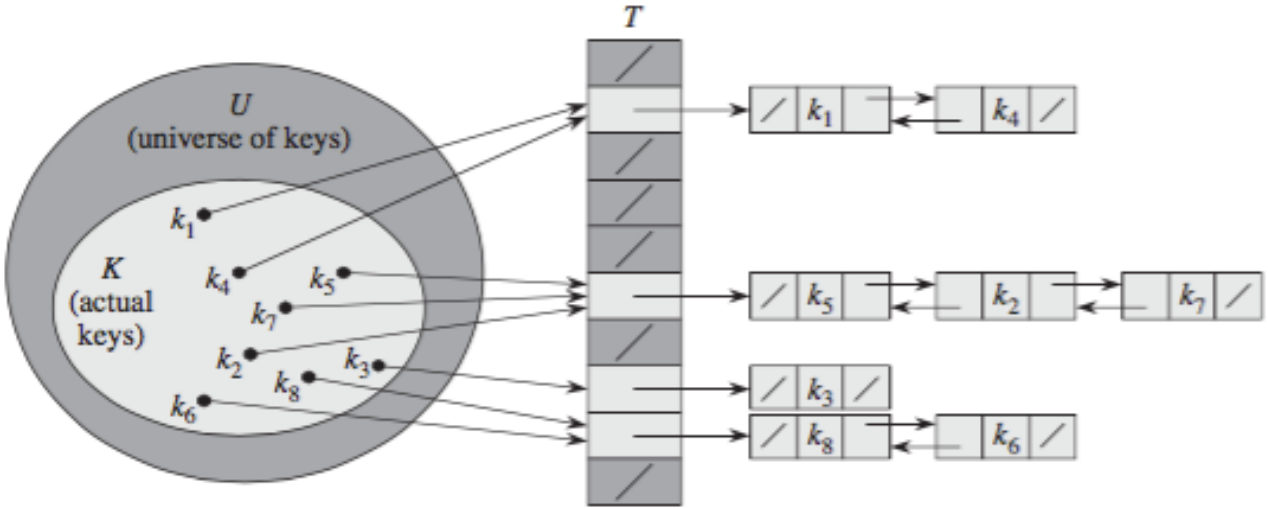
- Given an array, what is the index of a value k?
- Ordinarily, we go through the array to find the index. O(n)
- But with direct addressing, we are able to find the index with O(1) complexity.
- Hash table is a direct-addressing data structure based on hash function and linked list.

Using a hash function h to map keys to hash-table slots. Because keys k_2 and k_5 map to the same slot, they collide.



Hash Table (2 of 2)

- Hash table is a direct-addressing data structure based on hash function and linked list.



MONTCLAIR
STATE UNIVERSITY